

Object-Oriented Programming Report

Pac-Man Game Project

Mehdi Talebikhatir

Matricola: 558948

Professor: Salvatore Distefano
Universita degli Studi di Messina

Contents

1	Abstract	3
2	Project Introduction and Overview	3
3	Inheritance	4
4	Encapsulation	4
5	Information Hiding	5
6	Abstraction	6
7	Polymorphism	6
7.1	Polymorphism by Inclusion	7
7.2	Method Overloading	7
7.3	Parametric Polymorphism	8
7.4	Polymorphism by Coercion and Type Casting	9
8	Interface Implementation	9
9	Composition	10
10	Subtyping	11
11	Exception Handling	11
12	Customization	13
12.1	Choosing the Level	13
12.2	Save and Load	13
12.3	Sound Choice	14
12.4	Leaderboard	14
13	Open/Closed Principle	15
14	Conclusion	16

1 Abstract

This report describes the Object-Oriented Programming concepts used in a Java Pac-Man game project. The project is organized around reusable classes for the game loop, map, entities, tiles, states, input, graphics, sound, and file management. The implementation uses inheritance, encapsulation, abstraction, polymorphism, interfaces, composition, subtyping, exception handling, customization, and the Open/Closed Principle. Each concept is explained with code snippets taken from the project.

2 Project Introduction and Overview

The project is a desktop Pac-Man-style game written in Java. The player moves through a tile-based map, eats dots, avoids ghosts, changes levels, saves and loads game progress, and records leaderboard scores. The code is divided into packages for entities, moving creatures, tiles, states, graphics, and input keys.

The main game class owns the main loop, window, input handlers, score, current level, and file manager:

```
public class Game implements Runnable {
    private GameWindow wnd;
    private final KeyManager key;
    private final Mouse mouse;
    private State menuState;
    private int level;
    private final GameFileManager gameFileManager;

    public Game(String title, int width, int height,
                GameFileManager gameFileManager) {
        key = new KeyManager();
        mouse = new Mouse();
        this.gameFileManager = gameFileManager;

        int[] loadedData = gameFileManager.loadGame();
        this.level = loadedData[0];
        this.score = loadedData[2];
    }
}
```

The design separates responsibilities: **Game** controls execution, **Map** manages the level layout, **EntityManager** manages active objects, **State** represents screens, and **Tile** represents map elements.

3 Inheritance

Inheritance allows a class to reuse fields and behavior from a parent class. In this project, the entity hierarchy starts from the abstract class `Entity`. Moving game objects inherit from `Creature`, and concrete objects such as `Player` and ghosts inherit from `Creature`.

```
public abstract class Entity {
    public abstract void Update() throws InvalidFileException;
    public abstract void Draw(Graphics g);
    public abstract void die();
}

public abstract class Creature extends Entity {
    public Creature(Handler handler, double x, double y) {
        this(handler, x, y, DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }
}

public class Player extends Creature implements Damageable {
    public Player(Handler handler, float x, float y) {
        super(handler, x, y);
    }
}
```

The player class receives movement, position, collision, and size behavior from the creature and entity base classes. This avoids duplicating common entity behavior in every moving object.

4 Encapsulation

Encapsulation means keeping data inside an object and accessing it through controlled methods. The `Entity` class keeps position, size, direction, and state private, then exposes getters and setters.

```
public abstract class Entity {
    private double x, y;
    private String direction = "right";
    private final Handler handler;
    private int width, height;
    private boolean active = true;

    public double getX() {
        return x;
    }
}
```

```

    }

    public void setx(double x) {
        this.x = x;
    }

    public boolean isActive() {
        return active;
    }

    public void setActive(boolean active) {
        this.active = active;
    }
}

```

Other classes cannot directly change `x`, `y`, or `active`. They must use methods such as `setx()`, `sety()`, or `setActive()`, which keeps object state organized and easier to maintain.

5 Information Hiding

Information hiding means hiding internal implementation details that other classes do not need to know. In `Creature`, movement is exposed through the public method `move()`, while the details of horizontal and vertical movement are hidden in private methods.

```

public abstract class Creature extends Entity {
    public void move() {
        moveX();
        moveY();
    }

    private void moveX() {
        if (xMove > 0) {
            int tx = (int) (getX() + xMove + getSolidArea().x
                            + getSolidArea().width) / Tile.getTileWidth();
            if (collision(tx, (int) getY() / Tile.getTileHeight())) {
                setx(getX() + xMove);
            }
        }
    }

    private void moveY() {

```

```

        if (yMove > 0) {
            int ty = (int) (getY() + yMove + getSolidArea().height)
                / Tile.getTileHeight();
            if (collision((int) getX() / Tile.getTileWidth(), ty)) {
                setY(getY() + yMove);
            }
        }
    }
}

```

Users of `Creature` only call `move()`. They do not need to know how collision checks and tile coordinate calculations are performed.

6 Abstraction

Abstraction defines what an object must do without forcing every subclass to do it the same way. The `Entity` and `State` classes are abstract because they define common behavior for game objects and screens.

```

public abstract class State {
    private static State currentState = null;
    private final Handler handler;

    public static void setState(State state) {
        State.currentState = state;
    }

    public abstract void Update() throws InvalidFileException;
    public abstract void Draw(Graphics g);
}

```

Every state must implement `Update()` and `Draw()`, but each state decides its own behavior. For example, `MenuState`, `GameState`, `SoundState`, and `LeaderboardState` all share the same abstract interface while implementing different screens.

7 Polymorphism

Polymorphism allows one interface or parent type to refer to many concrete object types. The project uses several forms of polymorphism.

7.1 Polymorphism by Inclusion

Inclusion polymorphism happens when a parent reference points to a child object. The `EntityManager` stores all objects as `Entity`, even though they may be `Player`, `Dot`, `RedGhost`, `BlueGhost`, and other subclasses.

```
public class EntityManager {
    private ArrayList<Entity> entities;

    public void Update() throws InvalidFileException {
        Iterator<Entity> it = entities.iterator();
        while (it.hasNext()) {
            Entity e = it.next();
            e.Update();
            if (!e.isActive()) {
                it.remove();
            }
        }
    }

    public void Draw(Graphics g) {
        for (Entity e : entities) {
            e.Draw(g);
        }
    }
}
```

The loop does not need to know the exact class of each object. Java selects the correct overridden method at runtime.

7.2 Method Overloading

Method overloading occurs when methods or constructors have the same name but different parameter lists. The project uses overloaded constructors in `Creature`.

```
public abstract class Creature extends Entity {
    public Creature(Handler handler, double x, double y,
                   int width, int height) {
        super(handler, x, y, width, height);
    }

    public Creature(Handler handler, double x, double y) {
        this(handler, x, y, DEFAULT_WIDTH, DEFAULT_HEIGHT);
    }
}
```

```
}
```

The second constructor allows a creature to be created with default width and height, while the first constructor allows custom dimensions.

The project also overloads image loading:

```
public static BufferedImage LoadImage(String path)
    throws ImageNotFoundException {
    return ImageIO.read(ImageLoader.class.getResource(path));
}

public static BufferedImage LoadImage(String path, int width, int height)
    throws ImageNotFoundException {
    BufferedImage originalImage = LoadImage(path);
    BufferedImage resizedImage =
        new BufferedImage(width, height, originalImage.getType());
    return resizedImage;
}
```

7.3 Parametric Polymorphism

Parametric polymorphism is implemented using generics. The `Grid<T>` class can store objects of any type.

```
public class Grid<T> {
    private final List<List<T>> grid;

    public void setCell(int row, int col, T item) {
        if (isValid(row, col)) {
            grid.get(row).set(col, item);
        }
    }

    public T getCell(int row, int col) {
        if (isValid(row, col)) {
            return grid.get(row).get(col);
        }
        return null;
    }
}
```

In the map, this generic class is used specifically as a tile grid:

```
private Grid<Tile> gameTiles;
```



```
gameTiles = new Grid<>(height, width);
```

7.4 Polymorphism by Coercion and Type Casting

Coercion happens when Java converts one type into another compatible type. The project uses numeric casting when drawing objects, because entity positions are stored as `double` but drawing coordinates require integers.

```
public void Draw(Graphics g) {  
    BufferedImage image = directionImages.get(getDirection());  
    g.drawImage(image, (int) getX(), (int) getY(), null);  
}
```

The project also uses upcasting, where a subclass object is stored in a parent reference:

```
State gameState = new GameState(handler, this.level);  
State.setState(gameState);
```

Here, a `GameState` object is treated as a general `State`. This is safe because `GameState` is a subtype of `State`.

8 Interface Implementation

An interface defines behavior that a class promises to provide. The project defines a `Damageable` interface.

```
public interface Damageable {  
    void takeDamage(int amount);  
}  
  
public class Player extends Creature implements Damageable {  
    @Override  
    public void takeDamage(int amount) {  
        System.out.println("Player taking " + amount + " damage!");  
        die();  
    }  
}
```

The `Player` class promises to implement `takeDamage()`. This makes it possible to handle damage through the interface type.

The project also implements Java built-in input interfaces:

```
public class KeyManager implements KeyListener {  
    @Override
```

```

    public void keyPressed(KeyEvent e) {
        keys[e.getKeyCode()] = true;
    }
}

public class Mouse implements MouseListener, MouseMotionListener {
    @Override
    public void mouseMoved(MouseEvent e) {
        x = e.getX();
        y = e.getY();
    }
}

```

9 Composition

Composition means building complex objects by combining other objects. In this project, Game is composed of a window, keyboard manager, mouse manager, state objects, and file manager.

```

public class Game implements Runnable {
    private GameWindow wnd;
    private final KeyManager key;
    private final Mouse mouse;
    private State menuState;
    private final GameFileManager gameFileManager;

    public Game(String title, int width, int height,
                GameFileManager gameFileManager) {
        key = new KeyManager();
        mouse = new Mouse();
        this.gameFileManager = gameFileManager;
    }
}

```

The Map class is also composed of a tile grid and an entity manager:

```

public class Map {
    private Grid<Tile> gameTiles;
    private final EntityManager entityManager;

    public Map(Handler handler, String path)
        throws InvalidFileException {

```

```

        entityManager =
            new EntityManager(handler, new Player(handler, 100, 100));
        loadMap(path);
    }
}

```

This design keeps each object focused on a clear responsibility.

10 Subtyping

Subtyping means that an object of a child class can be used where its parent type or implemented interface is expected. The developer-side evidence is in the class declarations.

```

public class Player extends Creature implements Damageable {
    // Player is a Creature and is also Damageable.
}

public class RedGhost extends Creature {
    // RedGhost is a Creature.
}

public class MenuState extends State {
    // MenuState is a State.
}

public class Block extends Tile {
    // Block is a Tile.
}

```

These declarations define the subtype relationships. Because `Player`, `RedGhost`, `MenuState`, and `Block` are subtypes, the rest of the project can use them through general parent types such as `Entity`, `Creature`, `State`, or `Tile`.

11 Exception Handling

Exception handling is used to detect and manage errors such as missing files, invalid map data, and missing images. The project defines custom exception classes.

```

public class InvalidFileException extends Exception {
    InvalidFileException(String message) {
        super(message);
    }
}

```

```

}

public class ImageNotFoundException extends Exception {
    ImageNotFoundException(String message) {
        super(message);
    }
}

```

The ReadMap class throws a custom exception if the map file cannot be read:

```

public static String loadFile(String path)
    throws InvalidFileException {
    StringBuilder builder = new StringBuilder();
    try {
        BufferedReader b = new BufferedReader(new FileReader(path));
        String line;
        while ((line = b.readLine()) != null) {
            builder.append(line).append("\n");
        }
        b.close();
    } catch (Exception e) {
        throw new InvalidFileException("The file was not found!");
    }
    return builder.toString();
}

```

The GameFileManager class also catches file errors and provides default values, so the game can start even if no save file exists.

```

public int[] loadGame() {
    int[] data = new int[3];
    try (BufferedReader reader =
        new BufferedReader(new FileReader(FILE_NAME))) {
        data[0] = Integer.parseInt(reader.readLine());
        data[1] = Integer.parseInt(reader.readLine());
        data[2] = Integer.parseInt(reader.readLine());
    } catch (IOException | NumberFormatException
        | NullPointerException e) {
        data[0] = 1;
        data[1] = 5;
        data[2] = 0;
    }
    return data;
}

```

12 Customization

In this report, customization means the parts of the program that are not completely fixed in the code and can change based on user choice or saved user data. The current project does not implement choosing a different player character. The player controls one Pac-Man player object. However, the project does provide customization through level selection, sound choice, save/load, and leaderboard persistence.

12.1 Choosing the Level

The player can choose which level to start from in the level/about screen. The selected option changes the active `GameState`.

```
if (getHandler().getMouse().pressed_left()
    && level1.contains(getHandler().getMouse().getX(),
                      getHandler().getMouse().getY())) {
    State.setState(new GameState(getHandler(), 1));
}

if (getHandler().getMouse().pressed_left()
    && level4.contains(getHandler().getMouse().getX(),
                      getHandler().getMouse().getY())) {
    State.setState(new GameState(getHandler(), 4));
}
```

This means the level is selected by the player instead of being only one fixed value.

12.2 Save and Load

Save and load are also customization-related because the game adapts to the player's previous progress. The saved level, lives, and score are loaded when the user chooses to continue.

```
if (getHandler().getMouse().pressed_left()
    && load.contains(getHandler().getMouse().getX(),
                    getHandler().getMouse().getY())) {
    int[] savedData = getHandler().getGameFileManager().loadGame();
    getHandler().getGame().setLevel(savedData[0]);
    getHandler().getGame().setScoreValue(savedData[2]);
    State.setState(new GameState(getHandler(), savedData[0]));
}
```

```
}
```

The save file stores progress:

```
public void saveGame(int level, int lives, int score) {
    try (BufferedWriter writer =
        new BufferedWriter(new FileWriter(FILE_NAME))) {
        writer.write(level + "\n");
        writer.write(lives + "\n");
        writer.write(score + "\n");
    } catch (IOException e) {
        System.err.println("Error saving game state: "
            + e.getMessage());
    }
}
```

12.3 Sound Choice

The user can open the sound state and choose whether sound should be active.

```
if (getHandler().getMouse().pressed_left()
    && sound.contains(getHandler().getMouse().getX(),
        getHandler().getMouse().getY())) {
    State.setState(new SoundState(getHandler()));
}
```

The sound class stores the sound status:

```
public class Sound {
    private int Sound_off = 0;

    public boolean isActive() {
        return Sound_off == 0;
    }

    public void setSoundOff(int sound_off) {
        Sound_off = sound_off;
    }
}
```

12.4 Leaderboard

The leaderboard stores user scores and keeps the highest scores. This is another form of user-specific customization because the displayed leaderboard depends on previous game

results.

```
public void saveLeaderboardScore(int score) {
    List<Integer> scores = loadLeaderboardScores();
    scores.add(score);
    Collections.sort(scores, Collections.reverseOrder());

    try (BufferedWriter writer =
        new BufferedWriter(new FileWriter(LEADERBOARD_FILE_NAME))) {
        int limit = Math.min(MAX_LEADERBOARD_SCORES, scores.size());
        for (int i = 0; i < limit; i++) {
            writer.write(String.valueOf(scores.get(i)));
            writer.newLine();
        }
    } catch (IOException e) {
        System.err.println("Error saving leaderboard: "
            + e.getMessage());
    }
}
```

13 Open/Closed Principle

The Open/Closed Principle says that software should be open for extension but closed for modification. In this project, many parts can be extended by adding new subclasses instead of rewriting the main loops.

For example, a new tile type can extend `Tile` and override `IsSolid()`:

```
public class Block extends Tile {
    public Block(int idd) {
        super(Assets.getBlock(), idd);
    }

    @Override
    public boolean IsSolid() {
        return true;
    }
}
```

The rest of the map can still draw and check tiles through the `Tile` type:

```
public Tile getTile(int x, int y) {
    if (gameTiles != null) {
        return gameTiles.getCell(y, x);
    }
}
```

```
}  
    return Tile.getBlock();  
}
```

The same idea applies to entities and states. A new enemy can extend **Creature**, and a new screen can extend **State**. The main update and draw loops can continue using parent references:

```
for (Entity e : entities) {  
    e.Draw(g);  
}  
  
if (State.getState() != null) {  
    State.getState().Update();  
}
```

This shows that the project is designed so new behavior can be added through extension.

14 Conclusion

The Pac-Man project demonstrates the main principles of Object-Oriented Programming in Java. Inheritance and abstraction organize shared behavior, encapsulation and information hiding protect internal data, polymorphism allows the game loop to work with many object types, interfaces define required behavior, and composition builds the game from focused components. The project also uses custom exceptions, user customization through level selection and save/load features, and an extensible design that supports the Open/Closed Principle.